# AI Agent Starter Kit

Multi-Agent Digital Product Automation Platform

---

A Comprehensive Technical Guide

v1.0.0 | Production-Ready Architecture


1. Introduction & Overview
2. System Architecture
3. Orchestrator Agent
4. Researcher Agent
5. Creator Agent
6. Evaluator Agent
7. Seller Agent
8. Setup & Installation
9. Use Cases & Applications
10. Deployment Guide
11. Testing
12. Extending the System
13. Conclusion

# 1. Introduction & Overview

The AI Agent Starter Kit is a production-ready multi-agent system that automates the entire digital product lifecycle — from market research and content creation to quality evaluation and storefront listing. Built with Python and asyncio, it demonstrates how specialized AI agents can collaborate through a structured task ledger to deliver consistent, high-quality results.

## What This Guide Covers

This comprehensive guide walks through every component of the system:

- **System Architecture** — How the five agent types communicate and coordinate
- **Orchestrator Agent** — The manager-led dispatcher that owns the task ledger
- **Researcher Agent** — Market synthesis and product strategy generation
- **Creator Agent** — Skill-based content generation engine
- **Evaluator Agent** — Quality scoring with configurable thresholds
- **Seller Agent** — Storefront listing with HITL approval gates
- **Deployment** — Docker, CI/CD, and production operations
- **Testing** — Comprehensive test suite with pytest
- **Extensibility** — How to add agents, skills, and integrations

## Design Philosophy

The system follows three core principles:

1. **Manager-Led Dispatchers** — Each phase is owned by a coordinator that tracks progress through a structured ledger, ensuring no task is lost and every failure is recoverable.
2. **Human-in-the-Loop Governance** — High-risk or high-value decisions require explicit human approval before execution, preventing autonomous errors in production.
3. **Deterministic & Testable** — Every agent uses structured inputs and outputs with no hidden state, making the full pipeline testable, debuggable, and auditable.

## Target Audience

This guide is designed for:

- **Software engineers** building multi-agent AI systems
- **Technical founders** automating digital product creation
- **DevOps teams** deploying agent-based automation pipelines
- **Content creators** scaling production with AI assistance
- **Researchers** studying agent coordination patterns

# 2. System Architecture

## High-Level Pipeline

The system processes a user goal through four sequential phases. Each phase is owned by a specialized agent, with the Orchestrator managing the task ledger and handling failures.

| Phase | Agent | Output |
|---|---|---|
| 1. Research | ResearcherAgent | research_brief.json |
| 2. Creation | CreatorAgent | product_assets/ |
| 3. Evaluation | EvaluatorAgent | evaluation_report.json |
| 4. Publishing | SellerAgent | listing.json |

## Directory Structure

```
ai-agent-starter-kit/
├── agents/
│   ├── orchestrator.py    # Task ledger & pipeline dispatcher
│   ├── researcher.py      # Market research & strategy
│   ├── creator.py         # Content generation engine
│   ├── evaluator.py       # Quality scoring & compliance
│   └── seller.py          # Storefront listing & HITL
├── skills/
│   ├── ebook_builder.json
│   └── code_template.json
├── knowledge/
│   ├── pricing_governance.md
│   └── quality_standards.md
├── outputs/               # Generated products land here
├── tests/
│   ├── test_orchestrator.py
│   ├── test_researcher.py
│   ├── test_creator.py
│   ├── test_evaluator.py
│   └── test_seller.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .github/
│   └── workflows/
│       └── ci.yml
├── .env.example
└── README.md
```

## Data Flow

Each agent receives structured dictionaries as input and returns structured dictionaries as output. The Orchestrator passes outputs from one phase as inputs to the next, creating a unidirectional data flow that prevents circular dependencies and makes the pipeline easy to reason about.

## Concurrency Model

The system uses Python's asyncio for cooperative multitasking. While the pipeline phases run sequentially (each depends on the previous output), individual agents can perform I/O-bound operations concurrently using run_in_executor for file operations and async HTTP calls for external APIs.

# 3. Orchestrator Agent

The Orchestrator Agent is the central coordinator that manages the task ledger and dispatches work to specialized agents. It provides full traceability of every operation through structured task entries.

## Task Ledger Data Model

Each task in the ledger tracks: task ID, agent role, description, status, inputs/outputs, timestamps, retry count, and error information. This provides complete auditability of the pipeline execution.

```
from dataclasses import dataclass, field
from datetime import datetime, timezone
from enum import Enum
from typing import Any

class TaskStatus(str, Enum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"
    REJECTED = "rejected"

@dataclass
class TaskEntry:
    task_id: str
    agent_role: str
    description: str
    status: TaskStatus = TaskStatus.PENDING
    inputs: dict[str, Any] = field(default_factory=dict)
    outputs: dict[str, Any] = field(default_factory=dict)
    created_at: str = field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )
    updated_at: str = field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )
    retry_count: int = 0
    error: str | None = None


@dataclass
class ProductManifest:
    product_id: str
    title: str
    product_type: str
    status: str
    version: str
    local_path: str
    created_at: str = field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )
    metadata: dict[str, Any] = field(default_factory=dict)
```

## Orchestrator Initialization

```
class OrchestratorAgent:
    """
    Manager-Led Task Ledger & Dynamic Dispatcher.

    Workflow:
        1. Receive user goal
        2. Initialize task ledger
        3. Dispatch Researcher → Creator → Evaluator (loop) → Seller
        4. Consolidate outputs into ProductManifest
    """
```

```
    def __init__(
        self,
        outputs_dir: Path,
        researcher: ResearcherAgent | None = None,
        creator: CreatorAgent | None = None,
        evaluator: EvaluatorAgent | None = None,
        seller: SellerAgent | None = None,
    ):
        self.outputs_dir = outputs_dir
        self.researcher = researcher or ResearcherAgent(outputs_dir=outputs_dir)
        self.creator = creator or CreatorAgent(outputs_dir=outputs_dir)
        self.evaluator = evaluator or EvaluatorAgent(outputs_dir=outputs_dir)
        self.seller = seller or SellerAgent(outputs_dir=outputs_dir)
        self.ledger: list[TaskEntry] = []
        self.product_manifest: ProductManifest | None = None
```

## Ledger Helper Methods

The _add_task and _update_task methods provide a clean interface for managing the task ledger. Every state transition is logged and timestamped.

```
def _add_task(
    self,
    agent_role: str,
    description: str,
    inputs: dict[str, Any] | None = None,
) -> TaskEntry:
    task = TaskEntry(
        task_id=f"task-{uuid.uuid4().hex[:8]}",
        agent_role=agent_role,
        description=description,
        inputs=inputs or {},
    )
    self.ledger.append(task)
    logger.info("Task added to ledger: %s [%s]", task.task_id, agent_role)
    return task


def _update_task(
    self,
    task: TaskEntry,
    status: TaskStatus,
    outputs: dict[str, Any] | None = None,
    error: str | None = None,
) -> None:
    task.status = status
    task.updated_at = datetime.now(timezone.utc).isoformat()
    if outputs is not None:
        task.outputs = outputs
    if error is not None:
        task.error = error
    logger.info("Task updated: %s → %s", task.task_id, status.value)
```

## Pipeline Execution: Phase 1 — Research

The first phase dispatches the Researcher Agent with the user's goal. The task is tracked through IN_PROGRESS to COMPLETED (or FAILED) states.

```
async def run_pipeline(self, user_goal: str) -> ProductManifest:
    """Execute the full product creation pipeline for a given user goal."""
    logger.info("Orchestrator received goal: %s", user_goal)
    product_id = f"prod-{uuid.uuid4().hex[:6]}"

    # ■■ Phase 1: Research ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    research_task = self._add_task(
        agent_role="researcher",
        description="Market research & product strategy",
        inputs={"goal": user_goal, "product_id": product_id},
    )
```

```
self._update_task(research_task, TaskStatus.IN_PROGRESS)

try:
    research_output = await self.researcher.execute(
        goal=user_goal, product_id=product_id
    )
    self._update_task(
        research_task, TaskStatus.COMPLETED, outputs=research_output
    )
except Exception as exc:
    self._update_task(research_task, TaskStatus.FAILED, error=str(exc))
    logger.exception("Researcher failed")
    raise
```

## Phase 2 — Creation

```
# ■■ Phase 2: Creation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
create_task = self._add_task(
    agent_role="creator",
    description="Generate product assets",
    inputs={
        "research": research_output,
        "product_id": product_id,
    },
)
self._update_task(create_task, TaskStatus.IN_PROGRESS)

try:
    creation_output = await self.creator.execute(
        research=research_output, product_id=product_id
    )
    self._update_task(
        create_task, TaskStatus.COMPLETED, outputs=creation_output
    )
except Exception as exc:
    self._update_task(create_task, TaskStatus.FAILED, error=str(exc))
    logger.exception("Creator failed")
    raise
```

## Phase 3 — Evaluation

```
# ■■ Phase 3: Evaluation Loop ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
eval_task = self._add_task(
    agent_role="evaluator",
    description="Quality & compliance evaluation",
    inputs={
        "product_id": product_id,
        "research": research_output,
        "creation": creation_output,
    },
)
self._update_task(eval_task, TaskStatus.IN_PROGRESS)

try:
    eval_output = await self.evaluator.execute(
        product_id=product_id,
        research=research_output,
        creation=creation_output,
    )
    self._update_task(eval_task, TaskStatus.COMPLETED, outputs=eval_output)
except Exception as exc:
    self._update_task(eval_task, TaskStatus.FAILED, error=str(exc))
    logger.exception("Evaluator failed")
    raise
```

## Phase 4 — Selling

```
# ■■ Phase 4: Selling / Publishing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sell_task = self._add_task(
    agent_role="seller",
```

```
        description="Storefront listing & marketing",
        inputs={
            "product_id": product_id,
            "research": research_output,
            "creation": creation_output,
            "evaluation": eval_output,
        },
    )
    self._update_task(sell_task, TaskStatus.IN_PROGRESS)

    try:
        sell_output = await self.seller.execute(
            product_id=product_id,
            research=research_output,
            creation=creation_output,
            evaluation=eval_output,
        )
        self._update_task(sell_task, TaskStatus.COMPLETED, outputs=sell_output)
    except Exception as exc:
        self._update_task(sell_task, TaskStatus.FAILED, error=str(exc))
        logger.exception("Seller failed")
        raise
```

## Ledger Status Transitions

Tasks move through the following states:

- **PENDING** — Task created but not yet started
- **IN_PROGRESS** — Agent actively executing
- **COMPLETED** — Successfully finished with outputs
- **FAILED** — Error occurred; check error field
- **REJECTED** — HITL approval denied

# 4. Researcher Agent

The Researcher Agent synthesizes market research and defines product strategy. It takes a user goal and produces a comprehensive research brief including target buyer personas, pricing recommendations, and competitive analysis.

## Agent Class Definition

```
class ResearcherAgent:
    """
    Synthesizes market research and defines product strategy.

    In a production deployment, this agent would call out to:
    - web_search for competitor analysis
    - Composio tools for marketplace data
    - Reddit/Twitter APIs for demand signals

    For this implementation, it uses a deterministic synthesis approach
    that can be extended with live data sources.
    """

    def __init__(self, outputs_dir: Path, knowledge_dir: Path | None = None):
        self.outputs_dir = outputs_dir
        # Default to knowledge directory relative to this file
        if knowledge_dir is None:
            self.knowledge_dir = Path(__file__).resolve().parent.parent / "knowledge"
        else:
            self.knowledge_dir = knowledge_dir
```

## Execute Method

```
async def execute(self, goal: str, product_id: str) -> dict[str, Any]:
    """
    Execute the research phase.

    Args:
        goal: The user's product goal (e.g., "Build a FastAPI SaaS starter kit")
        product_id: Unique identifier for this product

    Returns:
        Dictionary containing:
            - title: Product title
            - product_type: ebook | code_template | prompt_library | etc.
            - target_buyer_persona: Description of ideal buyer
            - problem_statement: The problem being solved
            - key_features: List of core features/benefits
            - chapter_outline: For ebooks, list of chapter titles
            - price_recommendation: Suggested price
            - pricing_tier: starter | professional | premium | enterprise
            - demand_score: 1-10 estimated market demand
            - uniqueness_score: 1-10 differentiation from competitors
            - competitive_analysis: Brief competitor landscape summary
    """
    logger.info("Researcher synthesizing strategy for: %s", goal)

    # Load pricing governance for informed price recommendations
    pricing_rules = self._load_pricing_governance()

    # Synthesize research (deterministic; extend with live APIs)
    research_output = self._synthesize_research(goal, pricing_rules)

    # Persist research brief
    await self._save_research_brief(product_id, research_output)

    logger.info(
```

```
            "Research complete: %s (type=%s, price=$%.2f)",
            research_output["title"],
            research_output["product_type"],
            research_output["price_recommendation"],
        )
        return research_output
```

## Heuristic Helpers

The Researcher uses pattern matching to infer product type, buyer persona, and pricing from the goal text. These heuristics can be replaced with LLM calls or live API integrations in production.

```
    @staticmethod
    def _infer_product_type(goal_lower: str) -> str:
        if any(kw in goal_lower for kw in ["ebook", "guide", "book", "whitepaper", "handbook"]):
            return "ebook"
        if any(kw in goal_lower for kw in ["template", "boilerplate", "starter kit", "scaffold", "micro-saas", "
            return "code_template"
        if any(kw in goal_lower for kw in ["prompt library", "prompts", "prompt pack"]):
            return "prompt_library"
        if any(kw in goal_lower for kw in ["course", "curriculum", "tutorial series"]):
            return "course"
        if any(kw in goal_lower for kw in ["checklist", "worksheet", "toolkit"]):
            return "toolkit"
        return "ebook"  # default


    @staticmethod
    def _infer_buyer_persona(goal_lower: str) -> str:
        if any(kw in goal_lower for kw in ["beginner", "newbie", "getting started", "intro"]):
            return "Beginners and career-switchers entering the field"
        if any(kw in goal_lower for kw in ["senior", "lead", "architect", "enterprise"]):
            return "Senior developers and technical leads building production systems"
        if any(kw in goal_lower for kw in ["freelancer", "indie", "solopreneur", "solo"]):
            return "Indie hackers, freelancers, and solopreneurs building MVPs"
        if any(kw in goal_lower for kw in ["startup", "founder", "cto", "vp"]):
            return "Startup founders and CTOs looking to ship fast"
        return "Mid-level developers and technical professionals seeking practical skills"
```

## Pricing Inference

```
    @staticmethod
    def _infer_pricing(
        goal_lower: str,
        product_type: str,
        pricing_rules: dict[str, Any],
    ) -> tuple[str, float]:
        # Map product type and scope to tier
        enterprise_signals = ["enterprise", "team", "organization", "multi-tenant", "scale"]
        premium_signals = ["advanced", "complete", "comprehensive", "production", "micro-saas", "saas"]

        if any(kw in goal_lower for kw in enterprise_signals):
            return "enterprise", 299.0
        if any(kw in goal_lower for kw in premium_signals):
            return "premium", 79.0
        if product_type in ("ebook", "toolkit"):
            return "professional", 39.0
        if product_type == "code_template":
            return "professional", 49.0
        return "starter", 19.0


    @staticmethod
    def _estimate_demand(goal_lower: str, product_type: str) -> int:
        high_demand_keywords = [
            "ai", "llm", "chatgpt", "openai", "python", "fastapi",
            "react", "typescript", "saas", "micro-saas", "automation",
        ]
        score = 5  # baseline
        for kw in high_demand_keywords:
            if kw in goal_lower:
```

```
            score += 1
        if product_type in ("code_template", "ebook"):
            score += 1
        return min(score, 10)
```

Key responsibilities:

- Infer product type from goal keywords (ebook, code_template, prompt_library, etc.)
- Generate target buyer personas based on audience signals
- Apply pricing governance rules to recommend price points
- Estimate market demand and product uniqueness scores
- Build chapter outlines for long-form content
- Produce competitive landscape summaries

# 5. Creator Agent

The Creator Agent generates product assets based on research briefs and skill SOPs. It supports multiple product types through a pluggable skill system, generating everything from markdown documentation to full code scaffolds.

## Agent Class Definition

```
class CreatorAgent:
    """
    Generates product assets based on research briefs and skill SOPs.

    Workflow:
        1. Load the appropriate skill from skills/
        2. Execute each SOP step to generate content
        3. Call save_product_asset to persist files
        4. Return creation output with paths to all generated assets
    """

    def __init__(self, outputs_dir: Path, skills_dir: Path | None = None):
        self.outputs_dir = outputs_dir
        # Default to the skills directory relative to this file's location
        if skills_dir is None:
            self.skills_dir = Path(__file__).resolve().parent.parent / "skills"
        else:
            self.skills_dir = skills_dir
```

## Skill Loading

Skills are JSON files that define SOP steps for each product type. The Creator loads the appropriate skill based on the research brief.

```
def _load_skill(self, product_type: str) -> dict[str, Any] | None:
    """Load the skill definition for the given product type."""
    skill_map = {
        "ebook": "ebook_builder.json",
        "code_template": "code_template.json",
        "prompt_library": "ebook_builder.json",  # reuse ebook skill
        "course": "ebook_builder.json",
        "toolkit": "ebook_builder.json",
    }
    skill_file = skill_map.get(product_type)
    if skill_file is None:
        return None
    skill_path = self.skills_dir / skill_file
    if not skill_path.exists():
        logger.warning("Skill file not found: %s", skill_path)
        return None
    try:
        return json.loads(skill_path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as exc:
        logger.error("Failed to load skill: %s", exc)
        return None
```

## Ebook Skill Execution

The ebook skill generates: outline, front matter, body chapters, back matter, full draft, and cover brief. Each step produces a separate markdown file.

```
async def _execute_ebook_skill(
    self,
```

```
    skill: dict[str, Any] | None,
    research: dict[str, Any],
    product_dir: Path,
    product_id: str,
) -> list[str]:
    """Execute the ebook builder skill SOP."""
    files_created: list[str] = []
    loop = asyncio.get_running_loop()

    # Step 2: Structure & Outline
    outline_content = self._generate_outline_md(research)
    outline_path = product_dir / "outline.md"
    await loop.run_in_executor(
        None,
        lambda: outline_path.write_text(outline_content, encoding="utf-8"),
    )
    files_created.append(str(outline_path.resolve()))

    # Step 3: Front Matter
    front_matter = self._generate_front_matter(research)
    front_matter_path = product_dir / "front_matter.md"
    await loop.run_in_executor(
        None,
        lambda: front_matter_path.write_text(front_matter, encoding="utf-8"),
    )
    files_created.append(str(front_matter_path.resolve()))

    # Step 4: Body Chapters
    chapters = research.get("chapter_outline", [])
    full_body_parts: list[str] = []
    for i, chapter_info in enumerate(chapters, start=1):
        chapter_content = self._generate_chapter(
            chapter_info, research, chapter_num=i
        )
        chapter_path = product_dir / f"chapter_{i:02d}.md"
        await loop.run_in_executor(
            None,
            lambda p=chapter_path, c=chapter_content: p.write_text(c, encoding="utf-8"),
        )
        files_created.append(str(chapter_path.resolve()))
        full_body_parts.append(chapter_content)

    # Step 5: Back Matter
    back_matter = self._generate_back_matter(research)
    back_matter_path = product_dir / "back_matter.md"
    await loop.run_in_executor(
        None,
        lambda: back_matter_path.write_text(back_matter, encoding="utf-8"),
    )
    files_created.append(str(back_matter_path.resolve()))

    # Step 6: Full Draft (combined)
    full_draft_parts = [front_matter, "\n\n---\n\n".join(full_body_parts), back_matter]
    full_draft = "\n\n---\n\n".join(full_draft_parts)
    draft_path = product_dir / "source.md"
    await loop.run_in_executor(
        None,
        lambda: draft_path.write_text(full_draft, encoding="utf-8"),
    )
    files_created.append(str(draft_path.resolve()))

    # Step 9: Cover Brief
    cover_brief = self._generate_cover_brief(research)
    cover_brief_path = product_dir / "cover_brief.md"
    await loop.run_in_executor(
        None,
        lambda: cover_brief_path.write_text(cover_brief, encoding="utf-8"),
    )
    files_created.append(str(cover_brief_path.resolve()))

    return files_created
```

## Code Template Skill Execution

The code template skill generates a complete project scaffold including architecture docs, core modules, documentation, tests, and demo assets.

```
async def _execute_code_template_skill(
    self,
    skill: dict[str, Any] | None,
    research: dict[str, Any],
    product_dir: Path,
    product_id: str,
) -> list[str]:
    """Execute the code template builder skill SOP."""
    files_created: list[str] = []

    # Step 2: Architecture Design
    arch_content = self._generate_architecture_md(research)
    arch_path = product_dir / "architecture.md"
    arch_path.write_text(arch_content, encoding="utf-8")
    files_created.append(str(arch_path.resolve()))

    # Step 3: Scaffold
    scaffold_dir = product_dir / "scaffold"
    scaffold_dir.mkdir(exist_ok=True)
    scaffold_files = self._generate_scaffold(research, scaffold_dir)
    files_created.extend(scaffold_files)

    # Step 4: Core Modules
    src_dir = product_dir / "src"
    src_dir.mkdir(exist_ok=True)
    src_files = self._generate_core_modules(research, src_dir)
    files_created.extend(src_files)

    # Step 5: Documentation
    docs_dir = product_dir / "docs"
    docs_dir.mkdir(exist_ok=True)
    docs_files = self._generate_documentation(research, docs_dir)
    files_created.extend(docs_files)

    # Step 6: Tests
    tests_dir = product_dir / "tests"
    tests_dir.mkdir(exist_ok=True)
    test_files = self._generate_tests(research, tests_dir)
    files_created.extend(test_files)

    # Step 9: Demo Assets
    demo_content = self._generate_demo_assets(research)
    demo_path = product_dir / "demo_assets.md"
    demo_path.write_text(demo_content, encoding="utf-8")
    files_created.append(str(demo_path.resolve()))

    return files_created
```

## Scaffold Generation

```
@staticmethod
def _generate_scaffold(research: dict[str, Any], scaffold_dir: Path) -> list[str]:
    """Generate scaffold configuration files."""
    files: list[str] = []

    # README
    readme_content = "\n".join([
        f"# {research.get('title', 'Project')}",
        "",
        f"{research.get('problem_statement', '')}",
        "",
        "## Quick Start",
        "",
        "```bash",
        "pip install -r requirements.txt",
        "python -m src.main",
        "```",
        "",
    ])
    readme_path = scaffold_dir / "README.md"
```

```
        readme_path.write_text(readme_content, encoding="utf-8")
        files.append(str(readme_path.resolve()))

        # requirements.txt
        req_content = "\n".join([
            "fastapi>=0.100.0",
            "uvicorn[standard]>=0.20.0",
            "pydantic>=2.0.0",
            "httpx>=0.25.0",
            "python-dotenv>=1.0.0",
            "",
        ])
        req_path = scaffold_dir / "requirements.txt"
        req_path.write_text(req_content, encoding="utf-8")
        files.append(str(req_path.resolve()))

        # Dockerfile
        dockerfile_content = "\n".join([
            "FROM python:3.12-slim",
            "",
            "WORKDIR /app",
            "",
            "COPY requirements.txt .",
            "RUN pip install --no-cache-dir -r requirements.txt",
            "",
            "COPY src/ ./src/",
            "",
            'CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]',
            "",
        ])
        dockerfile_path = scaffold_dir / "Dockerfile"
        dockerfile_path.write_text(dockerfile_content, encoding="utf-8")
        files.append(str(dockerfile_path.resolve()))

        return files
```

Supported product types:

- **ebook** — Full-length guides with front matter, chapters, and back matter
- **code_template** — Production-ready project scaffolds with src/, tests/, docs/
- **prompt_library** — Curated prompt collections with expected outputs
- **course** — Structured curricula with learning objectives
- **toolkit** — Checklists, worksheets, and practical tools

# 6. Evaluator Agent

The Evaluator Agent scores generated products against quality standards using a weighted scoring methodology. Products scoring below the 85% threshold receive structured critique for revision.

## Agent Class Definition

```
class EvaluatorAgent:
    """
    Evaluates generated products against quality standards.

    Scoring methodology:
        - Content Quality (30%)
        - Structural Integrity (20%)
        - Market Viability (20%)
        - Technical Correctness (20%)
        - Compliance & Ethics (10%)

    Pass threshold: 85/100
    """

    # Weights aligned with knowledge/quality_standards.md
    WEIGHTS: dict[str, float] = {
        "content_quality": 0.30,
        "structural_integrity": 0.20,
        "market_viability": 0.20,
        "technical_correctness": 0.20,
        "compliance_ethics": 0.10,
    }

    PASS_THRESHOLD: float = 85.0

    def __init__(self, outputs_dir: Path, knowledge_dir: Path | None = None):
        self.outputs_dir = outputs_dir
        self.knowledge_dir = knowledge_dir or outputs_dir.parent / "knowledge"
```

## Execute Method

```
async def execute(
    self,
    product_id: str,
    research: dict[str, Any],
    creation: dict[str, Any],
) -> dict[str, Any]:
    """
    Execute the evaluation phase.

    Returns:
        Dictionary containing:
            - product_id: The product ID
            - total_score: 0-100 weighted score
            - passed: Whether the product meets the threshold
            - category_scores: Breakdown by category
            - critique: Structured feedback for revision (if failed)
            - recommendations: Improvement suggestions
    """
    logger.info("Evaluator scoring product: %s", product_id)

    product_dir = self.outputs_dir / product_id
    if not product_dir.exists():
        raise FileNotFoundError(f"Product directory not found: {product_dir}")

    # Read generated files for evaluation
    source_files = list(product_dir.glob("**/*.md"))
    source_content = await self._read_source_files(source_files)
    # Score each category
```

```
    category_scores = {
        "content_quality": self._score_content_quality(source_content, research),
        "structural_integrity": self._score_structural_integrity(source_content, creation),
        "market_viability": self._score_market_viability(research),
        "technical_correctness": self._score_technical_correctness(product_dir, creation),
        "compliance_ethics": self._score_compliance_ethics(source_content),
    }

    # Calculate weighted total
    total_score = sum(
        score * self.WEIGHTS[category]
        for category, score in category_scores.items()
    )
    total_score = round(total_score, 1)
    passed = total_score >= self.PASS_THRESHOLD

    evaluation_output: dict[str, Any] = {
        "product_id": product_id,
        "total_score": total_score,
        "passed": passed,
        "category_scores": category_scores,
        "threshold": self.PASS_THRESHOLD,
    }

    if not passed:
        evaluation_output["critique"] = self._generate_critique(category_scores)
        evaluation_output["recommendations"] = self._generate_recommendations(
            category_scores, total_score
        )
    else:
        evaluation_output["critique"] = None
        evaluation_output["recommendations"] = []

    # Save evaluation report
    await self._save_evaluation_report(product_id, evaluation_output)
    return evaluation_output
```

## Scoring Categories

| Category | Weight | Focus |
|---|---|---|
| Content Quality | 30% | Depth, accuracy, originality |
| Structural Integrity | 20% | Organization, formatting, hierarchy |
| Market Viability | 20% | Demand, differentiation, pricing |
| Technical Correctness | 20% | Code, tests, documentation |
| Compliance & Ethics | 10% | Attribution, safety, licensing |

## Content Quality Scoring

Content quality is assessed based on word count depth, structural indicators (headings, code blocks, lists, links), and topic coverage. Products with more than 10,000 words and rich formatting score highest.

```
@staticmethod
def _score_content_quality(source_content: str, research: dict) -> float:
    """Score content quality (0-100) based on depth, accuracy, and originality."""
    if not source_content:
        return 0.0
    word_count = len(source_content.split())
    score = 0.0

    # Word count depth
```

```
        if word_count > 10000:
            score += 30.0
        elif word_count > 5000:
            score += 25.0
        elif word_count > 2000:
            score += 18.0
        elif word_count > 500:
            score += 10.0
        else:
            score += 5.0

        # Structure indicators
        has_headings = "#" in source_content
        has_code_blocks = "```" in source_content
        has_lists = "- " in source_content
        has_links = "](" in source_content

        if has_headings:
            score += 15.0
        if has_code_blocks:
            score += 15.0
        if has_lists:
            score += 10.0
        if has_links:
            score += 10.0

        # Topic coverage (does it address the problem statement?)
        problem_keywords = research.get("problem_statement", "").lower().split()[:5]
        if any(kw in source_content.lower() for kw in problem_keywords if len(kw) > 4):
            score += 10.0

        return min(score, 100.0)
```

## Structural Integrity Scoring

```
    @staticmethod
    def _score_structural_integrity(source_content: str, creation: dict) -> float:
        """Score structural integrity (0-100) based on organization and formatting."""
        score = 0.0
        if not source_content:
            return 0.0

        # Files created count
        files_created = creation.get("files_created", [])
        if len(files_created) >= 8:
            score += 25.0
        elif len(files_created) >= 5:
            score += 20.0
        elif len(files_created) >= 3:
            score += 12.0
        else:
            score += 5.0

        # Heading hierarchy
        lines = source_content.split("\n")
        h1_count = sum(1 for line in lines if line.startswith("# "))
        h2_count = sum(1 for line in lines if line.startswith("## "))
        h3_count = sum(1 for line in lines if line.startswith("### "))

        if h1_count >= 3:
            score += 15.0
        if h2_count >= 5:
            score += 15.0
        if h3_count >= 10:
            score += 10.0

        # Consistent formatting
        if "---" in source_content:  # section breaks
            score += 10.0
        if "```" in source_content:  # code blocks
            score += 10.0
        if "|---" in source_content or "|-" in source_content:  # tables
            score += 5.0
        # Table of contents indicator
```

```
    if "table of contents" in source_content.lower():
        score += 10.0

    return min(score, 100.0)
```

## Revision Loop

Products scoring below 85/100 receive structured critique with actionable recommendations. The system supports up to 3 refinement loops where the Creator addresses feedback and the Evaluator re-scores the revised product.

# 7. Seller Agent

The Seller Agent implements Human-in-the-Loop (HITL) governance to prevent autonomous publishing of high-risk or high-value products. When certain conditions are met, the system pauses and requests human approval before proceeding.

## Agent Class Definition

```
class SellerAgent:
    """
    Lists products on the storefront with marketing copy.

    HITL Governance:
        - Price >= $50 → requires human approval
        - High-risk deployment → requires human approval
        - Regulated industries → requires human approval

    In dry-run mode, products are saved locally but not published externally.
    """

    HITL_PRICE_THRESHOLD: float = 50.0

    def __init__(
        self,
        outputs_dir: Path,
        pricing_rules: dict[str, Any] | None = None,
    ):
        self.outputs_dir = outputs_dir
        self.pricing_rules = pricing_rules or {}
        self.pending_approvals: list[dict[str, Any]] = []
```

## Execute Method

```
async def execute(
    self,
    product_id: str,
    research: dict[str, Any],
    creation: dict[str, Any],
    evaluation: dict[str, Any],
) -> dict[str, Any]:
    """
    Execute the selling phase.

    Returns:
        Dictionary containing:
            - product_id: The product ID
            - listing_title: Storefront title
            - description: Sales copy
            - price: Final price
            - tier: Pricing tier
            - hitl_required: Whether HITL approval was triggered
            - hitl_status: pending | approved | not_required
            - published: Whether the product was published
            - listing_url: URL or path to the listing
    """
    logger.info("Seller creating listing for: %s", product_id)

    title = research.get("title", "Untitled Product")
    price = research.get("price_recommendation", 29.0)
    tier = research.get("pricing_tier", "professional")

    # Generate sales copy
    sales_copy = self._generate_sales_copy(research, creation, evaluation)

    # Determine HITL requirement
```

```
    hitl_required = self._check_hitl_required(research, price)

    if hitl_required:
        logger.info("HITL approval required for %s (price=$%.2f)", product_id, price)
        hitl_request = self._create_hitl_request(
            product_id=product_id, title=title, price=price, tier=tier, research=research,
        )
        self.pending_approvals.append(hitl_request)
        return {
            "product_id": product_id,
            "listing_title": title,
            "description": sales_copy,
            "price": price,
            "tier": tier,
            "hitl_required": True,
            "hitl_status": "pending",
            "hitl_request": hitl_request,
            "published": False,
            "listing_url": None,
        }

    # No HITL required — publish directly
    listing = await self._publish_product(
        product_id=product_id, title=title,
        description=sales_copy, price=price,
        asset_path=str(self.outputs_dir / product_id),
    )
    return {
        "product_id": product_id,
        "listing_title": title,
        "description": sales_copy,
        "price": price,
        "tier": tier,
        "hitl_required": False,
        "hitl_status": "not_required",
        "published": True,
        "listing_url": listing.get("listing_url"),
    }
```

## HITL Triggers

The following conditions trigger HITL approval:

• **Price threshold** — Products priced at $50 or above

• **High-risk deployment** — References to production databases, API keys, credentials, Kubernetes, or cloud deployments

• **Regulated industries** — Healthcare/HIPAA, finance/PCI-DSS, legal compliance

## HITL Decision Logic

```
def _check_hitl_required(self, research: dict[str, Any], price: float) -> bool:
    """Determine if human approval is required before publishing."""
    # Price threshold
    hitl_price = self.pricing_rules.get("hitl_price_threshold", self.HITL_PRICE_THRESHOLD)
    if price >= hitl_price:
        return True

    # High-risk deployment check
    goal = research.get("problem_statement", "").lower()
    high_risk_signals = [
        "production database", "api keys", "credentials",
        "kubernetes", "aws", "gcp", "azure deployment",
    ]
    if any(signal in goal for signal in high_risk_signals):
        return True

    # Regulated industries
    regulated_signals = ["healthcare", "hipaa", "finance", "pci-dss", "legal compliance"]
```

```
        if any(signal in goal for signal in regulated_signals):
            return True

        return False
```

## Sales Copy Generation

The Seller Agent automatically generates marketing copy from research data, including: product title, target audience, problem statement, feature highlights with quality scores, and audience-specific positioning.

```
    @staticmethod
    def _generate_sales_copy(
        research: dict[str, Any],
        creation: dict[str, Any],
        evaluation: dict[str, Any],
    ) -> str:
        """Generate marketing sales copy for the product."""
        title = research.get("title", "Digital Product")
        persona = research.get("target_buyer_persona", "professionals")
        problem = research.get("problem_statement", "")
        features = research.get("key_features", [])
        word_count = int(creation.get("word_count", 0))
        score = evaluation.get("total_score", 0)

        lines = [
            f"# {title}",
            "",
            f"**Built for:** {persona}",
            "",
            "## The Problem",
            "",
            problem,
            "",
            "## What You Get",
            "",
        ]
        for feat in features:
            lines.append(f"- ■ {feat}")

        lines.extend([
            "",
            "## Quality Guaranteed",
            "",
            f"This product scored **{score:.0f}/100** on our quality evaluation, "
            f"covering {word_count:,d} words of expert content. "
            "Every claim is backed by practical examples and battle-tested patterns.",
            "",
            "## Who This Is For",
            "",
            f"This is designed specifically for {persona.lower()} who want a "
            "complete, production-ready solution without the trial-and-error.",
            "",
        ])
        return "\n".join(lines)
```

## Publishing Logic

```
    async def _publish_product(
        self, product_id: str, title: str,
        description: str, price: float, asset_path: str,
    ) -> dict[str, Any]:
        """Publish the product to the storefront.

        In dry-run mode, saves locally. In production, would integrate
        with Stripe, Airtable, or Composio for actual listing.
        """
        product_dir = self.outputs_dir / product_id
        product_dir.mkdir(parents=True, exist_ok=True)
        listing_data = {
```

```
        "product_id": product_id,
        "title": title,
        "description": description,
        "price": price,
        "asset_path": asset_path,
        "status": "published",
        "listing_url": f"/store/products/{product_id}",
    }

    # Save listing data locally
    listing_path = product_dir / "listing.json"
    listing_path.write_text(
        json.dumps(listing_data, indent=2, default=str), encoding="utf-8"
    )

    # Save sales copy
    sales_path = product_dir / "sales_copy.md"
    sales_path.write_text(description, encoding="utf-8")

    return listing_data
```

## Approval Workflow

1. Seller Agent evaluates HITL conditions
2. If triggered, creates a pending approval request
3. Product status set to pending — not published
4. Human reviews via approval URL
5. On approval: product publishes and listing goes live
6. On rejection: product returns to Creator for revision

# 8. Setup & Installation

## Prerequisites

- Python 3.12 or higher
- pip or uv package manager
- Git for version control
- Docker (optional, for containerized deployment)

## Quick Start

```
# Clone the repository
git clone https://github.com/your-repo/ai-agent-starter-kit.git
cd ai-agent-starter-kit

# Create virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment variables
cp .env.example .env
# Edit .env with your API keys

# Run the orchestrator
python -m agents.orchestrator --goal "Your product idea"

# Or use the Python API directly
python3 -c "
import asyncio
from agents.orchestrator import OrchestratorAgent
from pathlib import Path

async def main():
    orch = OrchestratorAgent(outputs_dir=Path("./outputs"))
    manifest = await orch.run_pipeline("Build a FastAPI SaaS starter kit")
    print(f"Product created: {manifest.product_id}")

asyncio.run(main())
"
```

## Requirements File

```
# Core
fastapi>=0.100.0
uvicorn[standard]>=0.20.0
pydantic>=2.0.0
pydantic-settings>=2.0.0
httpx>=0.25.0
python-dotenv>=1.0.0

# Async
asyncio>=3.4.3
aiofiles>=23.0.0

# Data
jsonschema>=4.0.0
pyyaml>=6.0
```

```
# Testing
pytest>=7.0.0
pytest-asyncio>=0.21.0
pytest-cov>=4.0.0
httpx>=0.25.0  # For AsyncClient

# Production
gunicorn>=21.0.0
uvloop>=0.17.0
```

## Environment Configuration

Create a .env file with the following variables:

```
# API Keys (for production data sources)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...

# Gumroad Integration
GUMROAD_ACCESS_TOKEN=...

# Output Configuration
OUTPUTS_DIR=./outputs
SKILLS_DIR=./skills
KNOWLEDGE_DIR=./knowledge

# Evaluation Thresholds
PASS_THRESHOLD=85.0
MAX_REFINEMENT_LOOPS=3

# HITL Configuration
HITL_PRICE_THRESHOLD=50.0
DRY_RUN=false

# Logging
LOG_LEVEL=INFO
LOG_FORMAT=json
```

## Running Tests

```
# Run full test suite
pytest tests/ -v

# Run with coverage
pytest tests/ --cov=agents --cov-report=html

# Run specific agent tests
pytest tests/test_orchestrator.py -v
pytest tests/test_evaluator.py -v

# Run with async support
pytest tests/ -v --asyncio-mode=auto
```

# 9. Use Cases & Applications

## 9.1 Digital Product Creation

The primary use case: automatically generate and publish digital products on marketplaces like Gumroad. From a single goal description, the system produces market research, content, quality evaluation, and storefront listing.

Example goals:

- "Build a FastAPI SaaS starter kit with auth and Stripe"
- "Create an AI prompt library for content marketers"
- "Write a comprehensive guide to Python asyncio"
- "Build a micro-saas boilerplate with Next.js and Prisma"

## 9.2 Content Agencies

Content agencies can use the system to scale production of ebooks, guides, and courses. The Evaluator ensures consistent quality, while the HITL gate gives agency leads final approval before client delivery.

## 9.3 Developer Tooling

Development teams can generate project scaffolds, documentation sites, and starter kits. The Creator's code_template skill produces production-ready structures with tests, CI/CD, and Docker configs.

## 9.4 Education & Training

The course product type generates structured curricula with learning objectives, chapter outlines, and practical exercises. Perfect for bootcamps, workshops, and self-paced learning programs.

## 9.5 Market Research Automation

Even without publishing, the Researcher Agent provides valuable market intelligence: buyer personas, pricing analysis, demand estimation, and competitive landscape summaries. Use it to validate product ideas before investing in development.

## 9.6 Internal Tooling

Adapt the system for internal documentation generation, onboarding guides, and knowledge base creation. The structured pipeline ensures consistent quality across all internal content.

# 10. Deployment Guide

## Dockerfile

The included Dockerfile uses Python 3.12-slim for minimal footprint. It installs dependencies first for better layer caching.

```
FROM python:3.12-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first for better caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY src/ ./src/
COPY agents/ ./agents/
COPY skills/ ./skills/
COPY knowledge/ ./knowledge/

# Create outputs directory
RUN mkdir -p outputs

# Run the application
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]

# Build and run:
# docker build -t ai-agent-kit .
# docker run -p 8000:8000 --env-file .env ai-agent-kit
```

## Docker Compose

The docker-compose.yml defines the full stack including optional Redis for task queuing and result caching.

```
version: "3.9"

services:
  agent-kit:
    build: .
    ports:
      - "8000:8000"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - GUMROAD_ACCESS_TOKEN=${GUMROAD_ACCESS_TOKEN}
      - DRY_RUN=false
    volumes:
      - ./outputs:/app/outputs
      - ./skills:/app/skills
    restart: unless-stopped

  # Optional: Redis for task queue
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
```

```
volumes:
  redis_data:
```

## CI/CD Pipeline

The GitHub Actions workflow runs tests across multiple Python versions, performs linting and type checking, and deploys on main branch passes.

```
name: Test & Deploy

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12", "3.13"]

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Cache pip packages
        uses: actions/cache@v3
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Lint with ruff
        run: ruff check agents/ tests/

      - name: Type check with mypy
        run: mypy agents/

      - name: Run tests with coverage
        run: pytest tests/ --cov=agents --cov-report=xml --cov-fail-under=80

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          file: coverage.xml

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t ai-agent-kit:${{ github.sha }} .

      - name: Push to registry
        run: |
          docker tag ai-agent-kit:${{ github.sha }} registry/ai-agent-kit:latest
          docker push registry/ai-agent-kit:latest
```

## Production Considerations

- Set DRY_RUN=false only after thorough testing
- Configure HITL thresholds appropriate for your risk tolerance
- Monitor the task ledger for failed tasks and retry patterns
- Use structured logging (JSON) for production observability
- Set up alerts for HITL approval requests that stall
- Use environment-specific .env files (.env.production, .env.staging)
- Enable Sentry or similar for error tracking
- Configure log rotation for outputs/ directory

# 11. Testing

The system includes comprehensive tests for all agents using pytest. Tests cover the full pipeline, individual agent behavior, and edge cases.

## Orchestrator Tests

Tests verify that the pipeline creates a manifest, tracks all tasks, and properly records metadata.

```
import pytest
from pathlib import Path
from agents.orchestrator import OrchestratorAgent, TaskStatus


@pytest.fixture
def orchestrator(tmp_path):
    return OrchestratorAgent(outputs_dir=tmp_path)


@pytest.mark.asyncio
async def test_pipeline_creates_manifest(orchestrator):
    manifest = await orchestrator.run_pipeline("Test product idea")

    assert manifest.product_id.startswith("prod-")
    assert manifest.status == "published"
    assert manifest.version == "1.0.0"
    assert manifest.local_path is not None


@pytest.mark.asyncio
async def test_ledger_tracks_all_tasks(orchestrator):
    await orchestrator.run_pipeline("Test product idea")

    ledger = orchestrator.get_ledger()
    assert len(ledger) == 4  # research, create, evaluate, sell

    roles = [task["agent_role"] for task in ledger]
    assert "researcher" in roles
    assert "creator" in roles
    assert "evaluator" in roles
    assert "seller" in roles


@pytest.mark.asyncio
async def test_all_tasks_completed(orchestrator):
    await orchestrator.run_pipeline("Test product idea")

    ledger = orchestrator.get_ledger()
    for task in ledger:
        assert task["status"] == "completed", f"Task {task['task_id']} not completed"


@pytest.mark.asyncio
async def test_manifest_has_metadata(orchestrator):
    manifest = await orchestrator.run_pipeline("Test product idea")

    assert "research" in manifest.metadata
    assert "creation" in manifest.metadata
    assert "evaluation" in manifest.metadata
    assert "selling" in manifest.metadata
    assert "ledger" in manifest.metadata
```

## Evaluator Tests

Tests verify that the evaluator correctly scores products and generates appropriate critique for low-quality submissions.

```
import pytest
from pathlib import Path
from agents.evaluator import EvaluatorAgent


@pytest.fixture
def evaluator(tmp_path):
    return EvaluatorAgent(outputs_dir=tmp_path)


@pytest.mark.asyncio
async def test_evaluator_passes_quality_product(evaluator, tmp_path):
    product_id = "test-prod-001"
    product_dir = tmp_path / product_id
    product_dir.mkdir(parents=True, exist_ok=True)

    # Create a high-quality source file
    source_content = """
# Introduction
## Overview
This is a comprehensive guide.
- Feature 1
- Feature 2
```python
def example():
    pass
```
    """ * 100  # Repeat for word count

    (product_dir / "source.md").write_text(source_content)

    research = {
        "problem_statement": "comprehensive guide for developers",
        "demand_score": 8,
        "uniqueness_score": 7,
        "price_recommendation": 39.0,
        "pricing_tier": "professional",
        "target_buyer_persona": "Mid-level developers seeking practical skills",
    }
    creation = {
        "files_created": [str(product_dir / "source.md")] * 8,
        "word_count": 5000,
    }

    result = await evaluator.execute(product_id, research, creation)

    assert result["total_score"] >= 0
    assert result["total_score"] <= 100
    assert "category_scores" in result
    assert len(result["category_scores"]) == 5


@pytest.mark.asyncio
async def test_evaluator_fails_low_quality(evaluator, tmp_path):
    product_id = "test-prod-002"
    product_dir = tmp_path / product_id
    product_dir.mkdir(parents=True, exist_ok=True)

    (product_dir / "source.md").write_text("Short content.")

    research = {"problem_statement": "test", "demand_score": 3, "uniqueness_score": 3}
    creation = {"files_created": [], "word_count": 10}

    result = await evaluator.execute(product_id, research, creation)

    assert result["total_score"] < 85
    assert result["passed"] is False
    assert result["critique"] is not None
    assert len(result["recommendations"]) > 0
```

## Test Coverage Requirements

The CI pipeline enforces 80% code coverage minimum:

- Unit tests for all scoring methods
- Integration tests for the full pipeline
- Edge case tests for empty/malformed inputs
- Async tests using pytest-asyncio

# 12. Extending the System

## Adding New Agent Types

To add a new agent type:

1. Create a class with an async execute() method
2. Accept and return structured dictionaries
3. Register it in the Orchestrator's __init__
4. Add a corresponding phase in run_pipeline()
5. Add tests in tests/test_.py

## Custom Skills

Skills are JSON files in the skills/ directory that define SOP steps for the Creator Agent. Add new skills by creating a JSON file:

```
{
  "skill_id": "ebook_builder",
  "name": "Ebook Builder",
  "description": "Generates complete ebooks with front matter, chapters, and back matter",
  "version": "1.0.0",
  "steps": [
    {
      "step": 1,
      "action": "Load research brief",
      "input": "research_brief.json"
    },
    {
      "step": 2,
      "action": "Structure & Outline",
      "output": "outline.md",
      "description": "Generate chapter outline from research"
    },
    {
      "step": 3,
      "action": "Front Matter",
      "output": "front_matter.md",
      "description": "Title page, TOC, foreword, introduction"
    },
    {
      "step": 4,
      "action": "Body Chapters",
      "output": "chapter_NN.md",
      "description": "Generate each chapter with learning objectives"
    },
    {
      "step": 5,
      "action": "Back Matter",
      "output": "back_matter.md",
      "description": "Conclusion, glossary, references"
    },
    {
      "step": 6,
      "action": "Full Draft",
      "output": "source.md",
      "description": "Combine all sections into single file"
    },
    {
```

```
      "step": 7,
      "action": "Quality Check",
      "description": "Self-review against quality standards"
    },
    {
      "step": 8,
      "action": "Format Polish",
      "description": "Ensure consistent formatting throughout"
    },
    {
      "step": 9,
      "action": "Cover Brief",
      "output": "cover_brief.md",
      "description": "Generate cover design specifications"
    }
  ],
  "outputs": [
    "outline.md",
    "front_matter.md",
    "chapter_01.md",
    "chapter_02.md",
    "...",
    "back_matter.md",
    "source.md",
    "cover_brief.md"
  ]
}
```

## Integrating Live Data

The Researcher Agent's _synthesize_research method can be extended with live API calls:

- Web search APIs for real-time competitor analysis
- Reddit/Twitter APIs for demand signal detection
- Gumroad/Stripe APIs for pricing intelligence
- LLM APIs for content generation augmentation

## Custom Evaluation Criteria

Modify the EvaluatorAgent.WEIGHTS dictionary and scoring methods to match your quality standards. Add new scoring categories by extending the category_scores dict in execute().

## Adding New Product Types

To add a new product type (e.g., "video_script"):

1. Add the type to ResearcherAgent._infer_product_type()
2. Create a skill JSON in skills/
3. Add a _execute__skill method to CreatorAgent
4. Update the skill_map in _load_skill()
5. Add pricing tier logic in _infer_pricing()

# 13. Conclusion

The AI Agent Starter Kit provides a production-ready foundation for multi-agent digital product automation. Its modular architecture, structured task ledger, and HITL governance make it suitable for both experimentation and production use.

## Key Takeaways

1. **Separation of concerns** — Each agent has a single responsibility
2. **Structured data flow** — Dictionaries in, dictionaries out
3. **Full traceability** — The task ledger records every operation
4. **Quality gates** — Automated scoring with human oversight
5. **Extensible design** — Add agents, skills, and integrations easily

---

## Next Steps

1. Run the system locally with a test goal to see the full pipeline in action
2. Customize the skills directory with your own SOPs for your product types
3. Integrate live data sources into the Researcher Agent for real-time intelligence
4. Connect the Seller Agent to your Gumroad account for automated publishing
5. Set up monitoring and alerting for the task ledger and HITL approvals

For updates, issues, and contributions, visit the project repository.